Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Дисциплина
“Проектирование вычислительных систем”

Лабораторная работа №2

Последовательный интерфейс UART

Вариант №1

Выполнили: Малышев Никита Александрович,
Медведев Ярослав Александрович
Группа: P3412
Проверил: Пинкевич Василий Юрьевич

Санкт-Петербург
2026 год

Содержание

1. Задание 3
   1.1. Разработка программы для стенда с использованием интерфейса UART. 3
   1.2. Изучение логики работы приемопередатчика UART в STM32. 4
      1.2.1. Способ 1. 4
   1.3. Вариант №1. «Светофор» 4
2. Блок-схемы 5
3. Описание работы алгоритма 10
   3.1. Общая структура программы 10
   3.2. Драйвер UART 10
   3.3. Консоль 11
   3.4. Светофор 12
4. Исходный код 12
   4.1. Параметры платы (`board_config.h`) 13
   4.2. Критические секции (`critical_section.h`) 13
   4.3. Очередь байтов (заголовок) (`byte_queue.h`) 14
   4.4. Очередь байтов (реализация) (`byte_queue.c`) 14
   4.5. Драйвер UART (заголовок) (`uart_driver.h`) 16
   4.6. Драйвер UART (реализация) (`uart_driver.c`) 16
   4.7. Консоль (заголовок) (`console.h`) 20
   4.8. Консоль (реализация) (`console.c`) 20
   4.9. Светофор (заголовок) (`traffic_light.h`) 24
   4.10. Светофор (реализация) (`traffic_light.c`) 24
   4.11. Главная функция (`main.c`) 27
5. Вывод 28

1. Задание

1.1. Разработка программы для стенда с использованием интерфейса UART.

Разработать и реализовать два варианта драйвера UART для стенда SDK-1.1M: с использованием прерываний и в «режиме опроса» (без использования прерываний).

Драйверы должны базироваться на использовании функций стандартного драйвера UART, входящего в состав библиотеки HAL для STM32.

Функции драйвера, использующего прерывания, должны обеспечивать работу в «неблокирующем» режиме (возврат из функции происходит сразу же, без ожидания окончания приема/отправки), а также буферизацию как принимаемых, так и отправляемых данных в буфере типа «очередь» для исключения случайной потери данных. Размер буфера должен быть настраиваемым на этапе компиляции программы. Реализация буфера не должна быть завязана на особенности прикладной задачи программы.

В драйвере, не использующем прерывания, функция приема данных также должна быть «неблокирующей», то есть она не должна «повисать» до приема данных (которые могут никогда не поступить). При отсутствии данных она должна возвращать соответствующий статус.

При использовании режима «без прерываний» прерывания от соответствующего блока UART должны быть запрещены.

С использованием данных драйверов написать программу, которая реализует определенную вариантом функциональность. Для всех вариантов должно быть реализовано два режима работы программы: с использованием и без использования прерываний.

Каждый принимаемый стендом символ должен отсылаться обратно, чтобы он был выведен в консоли (так называемое «эхо»). Каждое новое сообщение от стенда должно выводиться с новой строки. Если вариант предусматривает работу с командами, то на каждую команду должен выводиться ответ, определенный в задании или «ОК», если ответ не требуется. Если введена команда, которая не поддерживается, должно быть выведено сообщение об этом.

Скорость работы интерфейса UART должна соответствовать указанной в варианте задания.

1.2. Изучение логики работы приемопередатчика UART в STM32.

Данная часть выполняется одним из двух способов на выбор исполнителей.

1.2.1. Способ 1.

Разработать собственную реализацию драйвера UART (не HAL и LL) и внедрить ее в разработанную ранее программу для стенда.

Требования к драйверу: – должны быть реализованы функции инициализации UART в режиме прерываний и в режиме опроса, приема и передачи данных, проверки состояния; – допускается использование определенной в HAL структуры регистров типа USART_TypeDef и define-определений; остальные структуры и функции для работы с UART/USART использовать нельзя, требуется разработать свои варианты.

При внедрении разработанного драйвера необходимо удалить из программы все обращения к функциям стандартных библиотек HAL и LL для настройки и использования UART, в том числе необходимо отключить инициализацию UART в графическом конфигураторе (файл *.ioc). Все должно выполняться с помощью функций разработанного драйвера.

Для защиты данной части требуется уметь детально объяснить, чем разработанный драйвер отличается от стандартных драйверов HAL и LL.

1.3. Вариант №1. «Светофор»

Доработать программу «светофор», добавив возможности отключения кнопки и задания величины тайм-аута (период, в течение которого горит красный).

Должны обрабатываться следующие команды, посылаемые через UART: – `?` – в ответ стенд должен прислать состояние, которое отображается в данный момент на светодиодах: `red`, `yellow`, `green`, `blinking green`, режим – `mode 1` или `mode 2` (см. далее), величину тайм-аута (сколько горит красный) – timeout …, и задействованы ли прерывания – символ `I` (interrupt) или `P` (polling); – `set mode 1` или `set mode 2` – установить режим работы светофора, когда обрабатываются или игнорируются нажатия кнопки; – `set timeout X`

– установить тайм-аут (`X` – длина периода в секундах); – `set interrupts on` или `set interrupts off` – включить или выключить прерывания.

Скорость обмена данными по UART – 57600 бит/с.

2. Блок-схемы

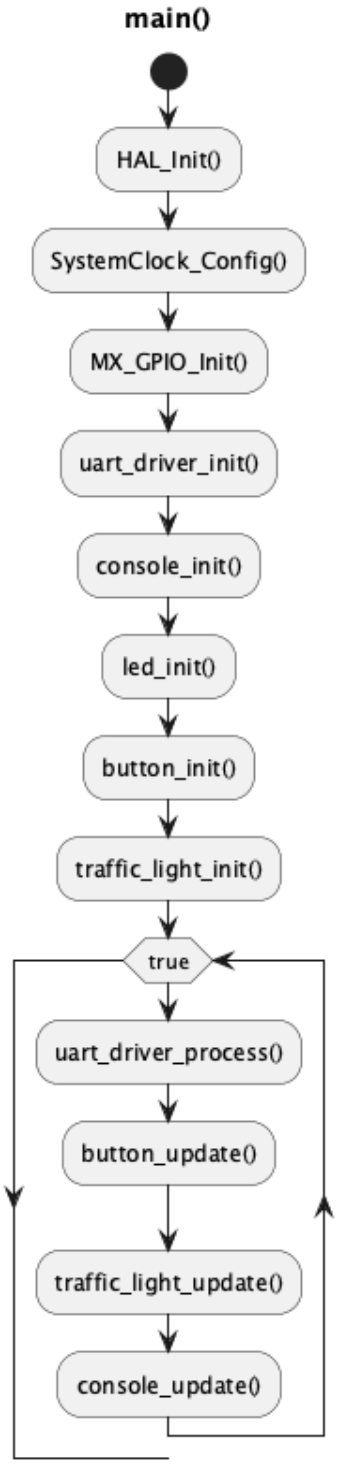


Рис. 1: Главная программа

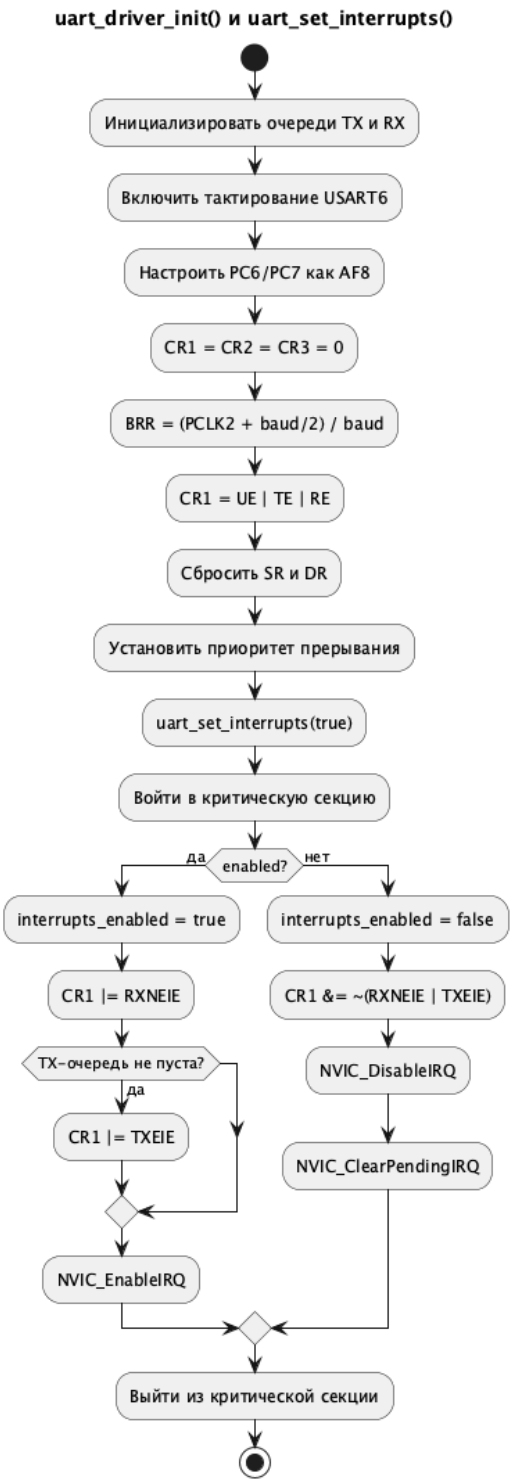


Рис. 2: Инициализация UART и настройка прерываний

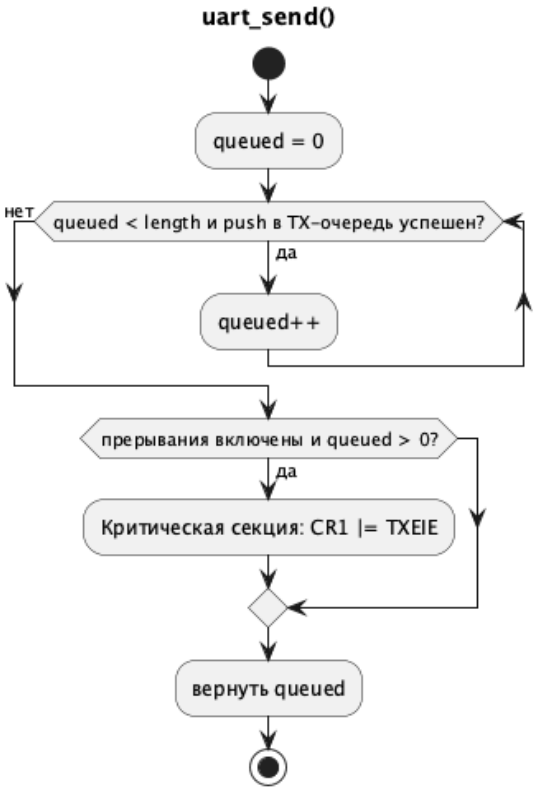


Рис. 3: Передача данных через UART



Рис. 4: Обработка UART в режиме опроса



Рис. 5: Обработчик прерывания UART

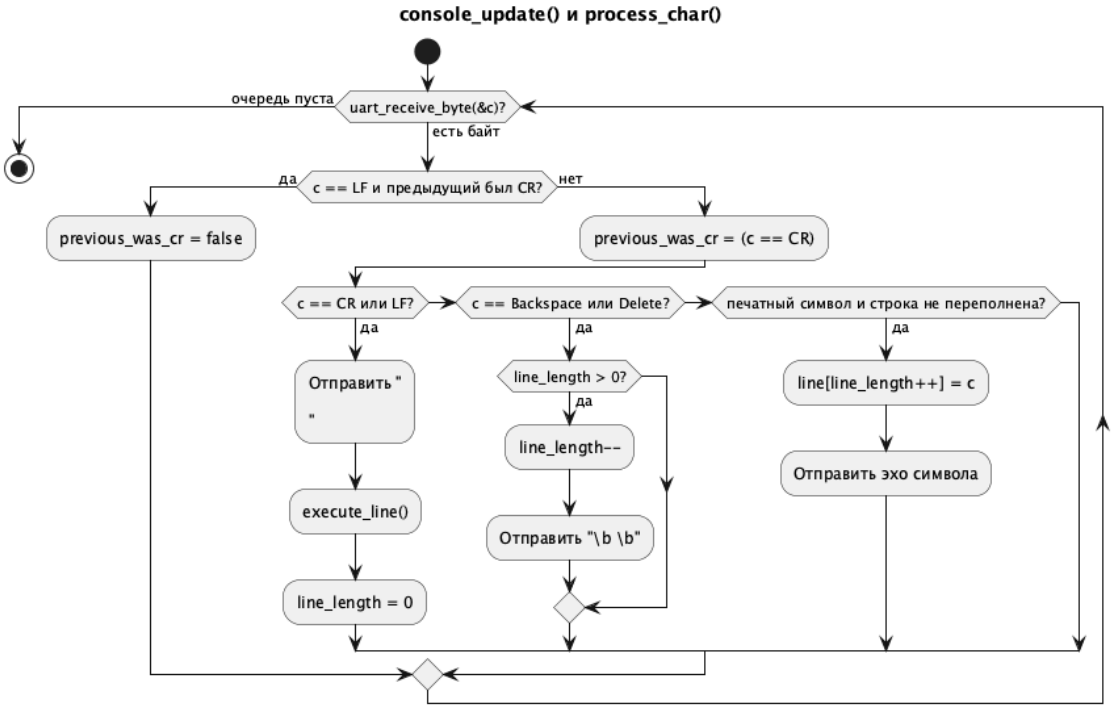


Рис. 6: Обработка символов консолью

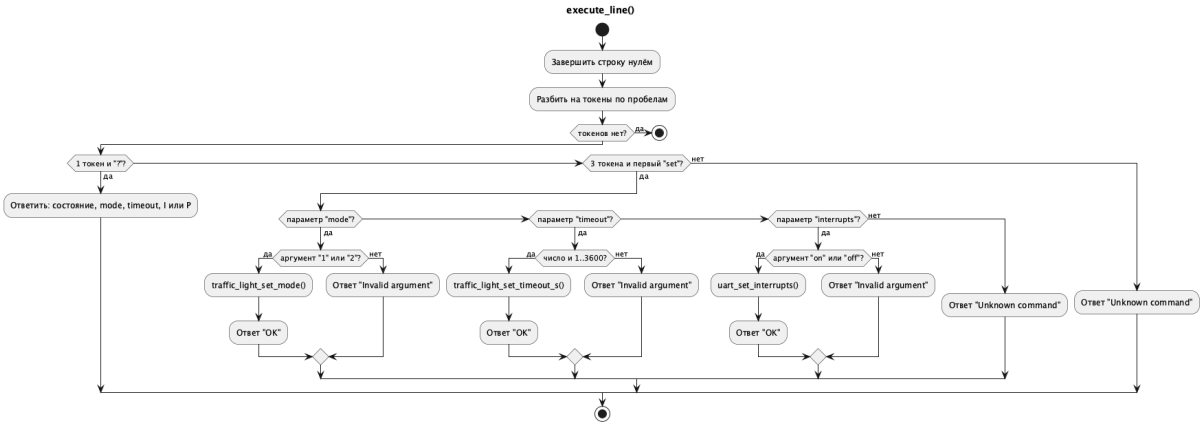


Рис. 7: Обработка введённой команды



Рис. 8: Обновление состояния светофора

3. Описание работы алгоритма

3.1. Общая структура программы

Программа построена по принципу «суперцикла» без операционной системы. Прикладная логика отделена от работы с аппаратурой и разбита на модули с минимальной взаимозависимостью:

- `uart_driver` – собственный драйвер UART, работающий напрямую с регистрами `USART_TypeDef` (без HAL и LL);
- `byte_queue` – универсальная кольцевая очередь байтов, не зависящая от прикладной задачи;
- `critical_section` – вход и выход из критической секции через `PRIMASK`;
- `console` – приём символов, эхо, разбор и выполнение команд;
- `traffic_light` – конечный автомат светофора;
- `led`, `button`, `gpio_driver` – драйверы светодиодов, кнопки и портов ввода-вывода;
- `board_config.h` – единственное место, где указаны порты, пины, номер USART и скорость обмена.

В функции `main()` после инициализации HAL, тактирования и портов вызываются `uart_driver_init()`, `console_init()`, `led_init()`, `button_init()` и `traffic_light_init()`. Затем в бесконечном цикле по очереди вызываются `uart_driver_process()`, `button_update()`, `traffic_light_update()` и `console_update()` (рис. 1). Все функции неблокирующие и возвращаются сразу, поэтому светофор, кнопка и консоль работают «одновременно».

3.2. Драйвер UART

Инициализация (рис. 2). Включается тактирование USART6 (`RCC->APB2ENR`), выводы PC6 и PC7 переводятся в режим альтернативной функции AF8, регистры `CR1`, `CR2` и `CR3` обнуляются (8 бит данных, без контроля чётности, 1 стоп-бит, передискретизация 16). Делитель скорости вычисляется как `BRR = (PCLK2 + baud / 2) / baud` с округлением; при частоте 16 МГц и скорости 57600 бит/с получается значение 278 (мантисса 17, дробная часть 6), погрешность около 0,1 %. После этого устанавливаются биты `UE`, `TE`, `RE` и по умолчанию включается режим прерываний.

Буферизация. Для приёма и передачи используются две очереди типа FIFO на статических массивах. Размеры задаются макросами `UART_TX_BUFFER_SIZE` и `UART_RX_BUFFER_SIZE` на этапе компиляции. Очередь хранит указатель на начало и счётчик элементов; все операции

выполняются в критической секции, что исключает гонки между главным циклом и обработчиком прерывания. При переполнении очереди передачи функция uart_send() возвращает число реально записанных байтов, лишние данные отбрасываются.

Режим прерываний (рис. 3 и 5). Прерывание по приёму (RXNEIE) включено постоянно. Обработчик читает регистр SR, затем DR (такое чтение сбрасывает флаги ORE, FE, NE), фиксирует ошибки в маске и кладёт корректный байт в очередь приёма. Прерывание по освобождению регистра передачи (TXEIE) включается только тогда, когда в очереди передачи есть данные: функция uart_send() записывает байты в очередь и устанавливает бит TXEIE. Обработчик на каждое прерывание TXE берёт очередной байт из очереди и записывает его в DR; когда очередь опустела, бит TXEIE сбрасывается. Таким образом, передача идёт «в фоне», а uart_send() возвращается сразу.

Режим опроса (рис. 4). Прерывания RXNEIE, TXEIE и вектор в NVIC запрещаются, ожидающий запрос прерывания сбрасывается. Функция uart_driver_process() за один вызов проверяет флаг RXNE (и при его наличии переносит байт в очередь приёма) и флаг TXE (и при его наличии записывает в DR один байт из очереди передачи). Ожидания в цикле нет, поэтому приём не «повисает»: при отсутствии данных uart_receive_byte() просто возвращает false.

Переключение режимов выполняется функцией uart_set_interrupts() внутри критической секции и сводится к изменению битов CR1 и состояния NVIC. Данные в очередях при переключении не теряются, так как они хранятся независимо от режима.

Проверка состояния. Функции uart_rx_available() и uart_tx_idle() сообщают о наличии принятых данных и об окончании передачи (очередь пуста и установлен флаг TC). Функция uart_take_errors() возвращает накопленные флаги ошибок (переполнение, кадровая ошибка, шум, чётность) и сбрасывает их.

3.3. Консоль

Функция console_update() в цикле забирает из очереди приёма все принятые байты и передаёт их в process_char() (рис. 6). Каждый печатный символ отправляется обратно (эхо) и добавляется в строку длиной до 47 символов. Символы Backspace и Delete удаляют последний символ строки и выводят последовательность \b \b. По символу конца строки (CR или LF; пара CR

LF обрабатывается как один перевод строки) выводится \r\n, после чего строка разбирается, поэтому ответ стенда всегда начинается с новой строки.

Функция execute_line() (рис. 7) разбивает строку на токены по пробелам и выбирает действие:

- ? – вывод состояния светофора, режима, тайм-аута и признака I или P, например red, mode 1, timeout 12, I;
- set mode 1 и set mode 2 – выбор режима работы кнопки;
- set timeout X – установка тайм-аута в секундах, допустимы значения от 1 до 3600;
- set interrupts on и set interrupts off – включение и выключение прерываний UART.

На корректную команду выводится OK, на неверный аргумент – Invalid argument, на неподдерживаемую команду – Unknown command. Пустая строка игнорируется.

3.4. Светофор

Светофор реализован конечным автоматом из четырёх состояний: красный, зелёный, мигающий зелёный и жёлтый (рис. 8). Время отсчитывается по HAL_GetTick() без задержек. Длительность красного равна заданному тайм-ауту (по умолчанию 12 с), длительность зелёного и короткого красного – четверти тайм-аута, мигающий зелёный горит 2 с с периодом переключения 250 мс, жёлтый – 2 с.

В режиме 1 нажатия кнопки обрабатываются: во время красного оставшееся время сокращается до четверти тайм-аута, во время мигающего зелёного и жёлтого устанавливается флаг, по которому следующий красный будет коротким, во время зелёного нажатие игнорируется. В режиме 2 нажатия кнопки игнорируются полностью. Новое значение тайм-аута применяется со следующего цикла красного. Антидребезг кнопки (30 мс) выполняется в модуле button.

4. Исходный код

Ниже приведены исходные тексты модулей, разработанных в рамках лабораторной работы. Код модулей led, button и gpio_driver, а также файлы, сгенерированные STM32CubeIDE, не изменялись.

4.1. Параметры платы (`board_config.h`)

```
#ifndef BOARD_CONFIG_H
#define BOARD_CONFIG_H

#include <stdbool.h>
#include "main.h"
#include "gpio_driver.h"

#define LED_GREEN_PORT        GPIOD
#define LED_GREEN_PIN         GPIO_PIN_13
#define LED_YELLOW_PORT       GPIOD
#define LED_YELLOW_PIN        GPIO_PIN_14
#define LED_RED_PORT          GPIOD
#define LED_RED_PIN           GPIO_PIN_15

#define LED_ON_LEVEL          true

#define BUTTON_PORT           GPIOC
#define BUTTON_PIN            GPIO_PIN_15
#define BUTTON_ACTIVE_LEVEL   false
#define BUTTON_PULL           PIN_PULL_UP

#define UART_INSTANCE         USART6
#define UART_IRQ_NUMBER       USART6_IRQn
#define UART_IRQ_HANDLER      USART6_IRQHandler
#define UART_IRQ_PRIORITY     0U
#define UART_BAUD_RATE        57600U
#define UART_CLOCK_ENABLE()   (RCC->APB2ENR |= RCC_APB2ENR_USART6EN)
#define UART_CLOCK_FREQ()     HAL_RCC_GetPCLK2Freq()

#define UART_PINS_PORT        GPIOC
#define UART_PINS             (GPIO_PIN_6 | GPIO_PIN_7)
#define UART_PINS_ALT_FUNC    8U

#endif
```

4.2. Критические секции (`critical_section.h`)

```
#ifndef CRITICAL_SECTION_H
#define CRITICAL_SECTION_H

#include <stdint.h>
#include "main.h"

static inline uint32_t critical_enter(void)
{
    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    return primask;
}

static inline void critical_exit(uint32_t primask)
```

```
{
    __set_PRIMASK(primask);
}

#endif
```

4.3. Очередь байтов (заголовок) (`byte_queue.h`)

```
#ifndef BYTE_QUEUE_H
#define BYTE_QUEUE_H

#include <stdbool.h>
#include <stdint.h>

typedef struct {
    uint8_t  *buffer;
    uint16_t  capacity;
    uint16_t  head;
    uint16_t  count;
} byte_queue_t;

void     byte_queue_init(byte_queue_t *queue, uint8_t *storage, uint16_t
capacity);
void     byte_queue_clear(byte_queue_t *queue);
bool     byte_queue_push(byte_queue_t *queue, uint8_t byte);
bool     byte_queue_peek(byte_queue_t *queue, uint8_t *byte);
bool     byte_queue_pop(byte_queue_t *queue, uint8_t *byte);
uint16_t byte_queue_count(byte_queue_t *queue);

#endif
```

4.4. Очередь байтов (реализация) (`byte_queue.c`)

```
#include "byte_queue.h"
#include "critical_section.h"

void byte_queue_init(byte_queue_t *queue, uint8_t *storage, uint16_t capacity)
{
    queue->buffer   = storage;
    queue->capacity = capacity;
    queue->head     = 0U;
    queue->count    = 0U;
}

void byte_queue_clear(byte_queue_t *queue)
{
    uint32_t primask = critical_enter();
    queue->head  = 0U;
    queue->count = 0U;
    critical_exit(primask);
}
```

```
bool byte_queue_push(byte_queue_t *queue, uint8_t byte)
{
    bool stored = false;
    uint32_t primask = critical_enter();

    if (queue->count < queue->capacity) {
        uint16_t tail = (uint16_t)((queue->head + queue->count) % queue-
>capacity);
        queue->buffer[tail] = byte;
        queue->count++;
        stored = true;
    }

    critical_exit(primask);
    return stored;
}

bool byte_queue_peek(byte_queue_t *queue, uint8_t *byte)
{
    bool available = false;
    uint32_t primask = critical_enter();

    if (queue->count > 0U) {
        *byte = queue->buffer[queue->head];
        available = true;
    }

    critical_exit(primask);
    return available;
}

bool byte_queue_pop(byte_queue_t *queue, uint8_t *byte)
{
    bool available = false;
    uint32_t primask = critical_enter();

    if (queue->count > 0U) {
        *byte = queue->buffer[queue->head];
        queue->head = (uint16_t)((queue->head + 1U) % queue->capacity);
        queue->count--;
        available = true;
    }

    critical_exit(primask);
    return available;
}

uint16_t byte_queue_count(byte_queue_t *queue)
{
    uint32_t primask = critical_enter();
    uint16_t count = queue->count;
    critical_exit(primask);
    return count;
}
```

4.5. Драйвер UART (заголовок) (`uart_driver.h`)

```
#ifndef UART_DRIVER_H
#define UART_DRIVER_H

#include <stdbool.h>
#include <stdint.h>

#ifndef UART_TX_BUFFER_SIZE
#define UART_TX_BUFFER_SIZE  256U
#endif

#ifndef UART_RX_BUFFER_SIZE
#define UART_RX_BUFFER_SIZE  64U
#endif

#define UART_ERROR_OVERRUN  (1UL << 0U)
#define UART_ERROR_FRAMING  (1UL << 1U)
#define UART_ERROR_NOISE    (1UL << 2U)
#define UART_ERROR_PARITY   (1UL << 3U)

void     uart_driver_init(void);
void     uart_driver_process(void);

void     uart_set_interrupts(bool enabled);
bool     uart_interrupts_enabled(void);

uint16_t uart_send(const uint8_t *data, uint16_t length);
uint16_t uart_send_string(const char *text);
bool     uart_receive_byte(uint8_t *byte);

bool     uart_rx_available(void);
bool     uart_tx_idle(void);
uint32_t uart_take_errors(void);

#endif
```

4.6. Драйвер UART (реализация) (`uart_driver.c`)

```
#include "uart_driver.h"
#include "board_config.h"
#include "byte_queue.h"
#include "critical_section.h"
#include <string.h>

#define RX_BAD_FRAME_FLAGS  (USART_SR_PE | USART_SR_FE | USART_SR_NE)

static uint8_t      tx_storage[UART_TX_BUFFER_SIZE];
static uint8_t      rx_storage[UART_RX_BUFFER_SIZE];
static byte_queue_t tx_queue;
static byte_queue_t rx_queue;

static volatile bool     interrupts_enabled;
```

```
static volatile uint32_t error_flags;

static void configure_pins(void)
{
    gpio_config_t config = {
        .mode     = PIN_MODE_ALT,
        .otype    = PIN_OTYPE_PUSH_PULL,
        .speed    = PIN_SPEED_VERY_HIGH,
        .pull     = PIN_PULL_NONE,
        .alt_func = UART_PINS_ALT_FUNC,
        .level    = PIN_LEVEL_KEEP
    };
    gpio_init(UART_PINS_PORT, UART_PINS, &config);
}

static uint32_t baud_divider(void)
{
    return (UART_CLOCK_FREQ() + (UART_BAUD_RATE / 2U)) / UART_BAUD_RATE;
}

static void configure_registers(void)
{
    UART_INSTANCE->CR1 = 0U;
    UART_INSTANCE->CR2 = 0U;
    UART_INSTANCE->CR3 = 0U;
    UART_INSTANCE->BRR = baud_divider();
    UART_INSTANCE->CR1 = USART_CR1_UE | USART_CR1_TE | USART_CR1_RE;

    (void)UART_INSTANCE->SR;
    (void)UART_INSTANCE->DR;
}

static void record_errors(uint32_t status)
{
    if ((status & USART_SR_ORE) != 0U) {
        error_flags |= UART_ERROR_OVERRUN;
    }
    if ((status & USART_SR_FE) != 0U) {
        error_flags |= UART_ERROR_FRAMING;
    }
    if ((status & USART_SR_NE) != 0U) {
        error_flags |= UART_ERROR_NOISE;
    }
    if ((status & USART_SR_PE) != 0U) {
        error_flags |= UART_ERROR_PARITY;
    }
}

static void receive_pending(void)
{
    uint32_t status = UART_INSTANCE->SR;

    if ((status & (USART_SR_RXNE | USART_SR_ORE)) == 0U) {
        return;
```

```
    }

    uint8_t byte = (uint8_t)UART_INSTANCE->DR;
    record_errors(status);

    if (((status & USART_SR_RXNE) != 0U) && ((status & RX_BAD_FRAME_FLAGS) ==
0U)) {
        (void)byte_queue_push(&rx_queue, byte);
    }
}

static bool transmit_next(void)
{
    uint8_t byte;

    if (((UART_INSTANCE->SR & USART_SR_TXE) != 0U) &&
        byte_queue_pop(&tx_queue, &byte)) {
        UART_INSTANCE->DR = byte;
        return true;
    }
    return false;
}

void UART_IRQ_HANDLER(void)
{
    receive_pending();

    if (((UART_INSTANCE->CR1 & USART_CR1_TXEIE) != 0U) &&
        ((UART_INSTANCE->SR & USART_SR_TXE) != 0U)) {
        if (!transmit_next()) {
            UART_INSTANCE->CR1 &= ~USART_CR1_TXEIE;
        }
    }
}

void uart_driver_init(void)
{
    byte_queue_init(&tx_queue, tx_storage, UART_TX_BUFFER_SIZE);
    byte_queue_init(&rx_queue, rx_storage, UART_RX_BUFFER_SIZE);

    UART_CLOCK_ENABLE();
    configure_pins();
    configure_registers();

    NVIC_SetPriority(UART_IRQ_NUMBER, UART_IRQ_PRIORITY);
    uart_set_interrupts(true);
}

void uart_set_interrupts(bool enabled)
{
    uint32_t primask = critical_enter();

    if (enabled) {
        interrupts_enabled = true;
```

```
        UART_INSTANCE->CR1 |= USART_CR1_RXNEIE;
        if (byte_queue_count(&tx_queue) > 0U) {
            UART_INSTANCE->CR1 |= USART_CR1_TXEIE;
        }
        NVIC_EnableIRQ(UART_IRQ_NUMBER);
    } else {
        interrupts_enabled = false;
        UART_INSTANCE->CR1 &= ~(USART_CR1_RXNEIE | USART_CR1_TXEIE);
        NVIC_DisableIRQ(UART_IRQ_NUMBER);
        NVIC_ClearPendingIRQ(UART_IRQ_NUMBER);
    }

    critical_exit(primask);
}

bool uart_interrupts_enabled(void)
{
    return interrupts_enabled;
}

void uart_driver_process(void)
{
    if (!interrupts_enabled) {
        receive_pending();
        (void)transmit_next();
    }
}

uint16_t uart_send(const uint8_t *data, uint16_t length)
{
    uint16_t queued = 0U;

    while ((queued < length) && byte_queue_push(&tx_queue, data[queued])) {
        queued++;
    }

    if (interrupts_enabled && (queued > 0U)) {
        uint32_t primask = critical_enter();
        UART_INSTANCE->CR1 |= USART_CR1_TXEIE;
        critical_exit(primask);
    }
    return queued;
}

uint16_t uart_send_string(const char *text)
{
    return uart_send((const uint8_t *)text, (uint16_t)strlen(text));
}

bool uart_receive_byte(uint8_t *byte)
{
    return byte_queue_pop(&rx_queue, byte);
}
```

```
bool uart_rx_available(void)
{
    return byte_queue_count(&rx_queue) > 0U;
}

bool uart_tx_idle(void)
{
    return (byte_queue_count(&tx_queue) == 0U) &&
           ((UART_INSTANCE->SR & USART_SR_TC) != 0U);
}

uint32_t uart_take_errors(void)
{
    uint32_t primask = critical_enter();
    uint32_t flags = error_flags;
    error_flags = 0U;
    critical_exit(primask);
    return flags;
}
```

4.7. Консоль (заголовок) (`console.h`)

```
#ifndef CONSOLE_H
#define CONSOLE_H

void console_init(void);
void console_update(void);

#endif
```

4.8. Консоль (реализация) (`console.c`)

```
#include "console.h"
#include "uart_driver.h"
#include "traffic_light.h"
#include <stdio.h>
#include <stdbool.h>
#include <stdint.h>
#include <string.h>

#define CONSOLE_LINE_MAX    48U
#define CONSOLE_TOKENS_MAX  4U
#define CONSOLE_NUMBER_MAX_DIGITS  7U
#define CHAR_BACKSPACE      0x08U
#define CHAR_DELETE         0x7FU
#define CHAR_PRINTABLE_MIN  0x20U
#define CHAR_PRINTABLE_MAX  0x7EU

static char    line[CONSOLE_LINE_MAX];
static uint8_t line_length;
static bool    previous_was_cr;
```

```
static void reply(const char *text)
{
    uart_send_string(text);
    uart_send_string("\r\n");
}

static void reply_result(bool success)
{
    reply(success ? "OK" : "Invalid argument");
}

static bool parse_unsigned(const char *text, uint32_t *value)
{
    uint32_t result = 0U;
    uint32_t digits = 0U;

    for (; *text != '\0'; text++, digits++) {
        if ((*text < '0') || (*text > '9') || (digits >=
CONSOLE_NUMBER_MAX_DIGITS)) {
            return false;
        }
        result = result * 10U + (uint32_t)(*text - '0');
    }

    *value = result;
    return digits > 0U;
}

static uint8_t tokenize(char *text, char *tokens[])
{
    uint8_t count = 0U;

    while (*text != '\0') {
        while (*text == ' ') {
            *text++ = '\0';
        }
        if (*text == '\0') {
            break;
        }
        if (count == CONSOLE_TOKENS_MAX) {
            return CONSOLE_TOKENS_MAX + 1U;
        }
        tokens[count++] = text;
        while ((*text != '\0') && (*text != ' ')) {
            text++;
        }
    }

    return count;
}

static void report_status(void)
{
    char text[80];
```

```
    snprintf(text, sizeof(text), "%s, mode %u, timeout %lu, %c",
             traffic_light_state_name(),
             (unsigned)traffic_light_get_mode(),
             (unsigned long)traffic_light_get_timeout_s(),
             uart_interrupts_enabled() ? 'I' : 'P');
    reply(text);
}

static void set_mode(const char *argument)
{
    if (strcmp(argument, "1") == 0) {
        traffic_light_set_mode(TRAFFIC_LIGHT_MODE_1);
        reply_result(true);
    } else if (strcmp(argument, "2") == 0) {
        traffic_light_set_mode(TRAFFIC_LIGHT_MODE_2);
        reply_result(true);
    } else {
        reply_result(false);
    }
}

static void set_timeout(const char *argument)
{
    uint32_t seconds;
    reply_result(parse_unsigned(argument, &seconds) &&
                 traffic_light_set_timeout_s(seconds));
}

static void set_interrupts(const char *argument)
{
    if (strcmp(argument, "on") == 0) {
        uart_set_interrupts(true);
        reply_result(true);
    } else if (strcmp(argument, "off") == 0) {
        uart_set_interrupts(false);
        reply_result(true);
    } else {
        reply_result(false);
    }
}

static void execute_set(const char *parameter, const char *argument)
{
    if (strcmp(parameter, "mode") == 0) {
        set_mode(argument);
    } else if (strcmp(parameter, "timeout") == 0) {
        set_timeout(argument);
    } else if (strcmp(parameter, "interrupts") == 0) {
        set_interrupts(argument);
    } else {
        reply("Unknown command");
    }
}
```

```
static void execute_line(void)
{
    char    *tokens[CONSOLE_TOKENS_MAX];
    uint8_t  count;

    line[line_length] = '\0';
    count = tokenize(line, tokens);

    if (count == 0U) {
        return;
    }

    if ((count == 1U) && (strcmp(tokens[0], "?") == 0)) {
        report_status();
    } else if ((count == 3U) && (strcmp(tokens[0], "set") == 0)) {
        execute_set(tokens[1], tokens[2]);
    } else {
        reply("Unknown command");
    }
}

static void process_char(uint8_t c)
{
    if ((c == '\n') && previous_was_cr) {
        previous_was_cr = false;
        return;
    }
    previous_was_cr = (c == '\r');

    if ((c == '\r') || (c == '\n')) {
        uart_send_string("\r\n");
        execute_line();
        line_length = 0U;
    } else if ((c == CHAR_BACKSPACE) || (c == CHAR_DELETE)) {
        if (line_length > 0U) {
            line_length--;
            uart_send_string("\b \b");
        }
    } else if ((c >= CHAR_PRINTABLE_MIN) && (c <= CHAR_PRINTABLE_MAX) &&
               (line_length < (CONSOLE_LINE_MAX - 1U))) {
        line[line_length++] = (char)c;
        uart_send(&c, 1U);
    }
}

void console_init(void)
{
    line_length     = 0U;
    previous_was_cr = false;
}

void console_update(void)
{
```

```
    uint8_t c;

    while (uart_receive_byte(&c)) {
        process_char(c);
    }
}
```

4.9. Светофор (заголовок) (`traffic_light.h`)

```
#ifndef TRAFFIC_LIGHT_H
#define TRAFFIC_LIGHT_H

#include <stdbool.h>
#include <stdint.h>

typedef enum {
    TRAFFIC_LIGHT_MODE_1 = 1U,
    TRAFFIC_LIGHT_MODE_2 = 2U
} traffic_light_mode_t;

void                 traffic_light_init(void);
void                 traffic_light_update(void);

const char          *traffic_light_state_name(void);

void                 traffic_light_set_mode(traffic_light_mode_t mode);
traffic_light_mode_t traffic_light_get_mode(void);

bool                 traffic_light_set_timeout_s(uint32_t seconds);
uint32_t             traffic_light_get_timeout_s(void);

#endif
```

4.10. Светофор (реализация) (`traffic_light.c`)

```
#include "traffic_light.h"
#include "led.h"
#include "button.h"
#include "main.h"

#define DEFAULT_RED_MS    12000U
#define MIN_TIMEOUT_S     1U
#define MAX_TIMEOUT_S     3600U
#define MS_PER_SECOND     1000U
#define GREEN_BLINK_MS    2000U
#define BLINK_HALF_MS     250U
#define YELLOW_MS         2000U

typedef enum {
    TL_RED,
    TL_GREEN,
    TL_GREEN_BLINK,
```

```
    TL_YELLOW
} tl_state_t;

static const char *const state_names[] = {
    [TL_RED]         = "red",
    [TL_GREEN]       = "green",
    [TL_GREEN_BLINK] = "blinking green",
    [TL_YELLOW]      = "yellow",
};

static tl_state_t           state;
static uint32_t             state_start;
static uint32_t             state_duration;
static uint32_t             last_blink_tick;
static bool                 short_red_pending;
static uint32_t             red_ms = DEFAULT_RED_MS;
static traffic_light_mode_t mode   = TRAFFIC_LIGHT_MODE_1;

static uint32_t quarter_of_red_ms(void)
{
    return red_ms / 4U;
}

static uint32_t elapsed(uint32_t now)
{
    return (uint32_t)(now - state_start);
}

static void enter_state(tl_state_t next, uint32_t now)
{
    state       = next;
    state_start = now;
    led_all_off();

    switch (next) {
    case TL_RED:
        led_set(LED_RED, true);
        state_duration    = short_red_pending ? quarter_of_red_ms() : red_ms;
        short_red_pending = false;
        break;
    case TL_GREEN:
        led_set(LED_GREEN, true);
        state_duration = quarter_of_red_ms();
        break;
    case TL_GREEN_BLINK:
        led_set(LED_GREEN, true);
        state_duration  = GREEN_BLINK_MS;
        last_blink_tick = now;
        break;
    case TL_YELLOW:
        led_set(LED_YELLOW, true);
        state_duration = YELLOW_MS;
        break;
    }
```

```
}

static void on_button(uint32_t now)
{
    switch (state) {
    case TL_GREEN:
        break;

    case TL_RED: {
        uint32_t spent = elapsed(now);
        if (spent < state_duration) {
            uint32_t left = state_duration - spent;
            if (left > quarter_of_red_ms()) {
                state_duration = spent + quarter_of_red_ms();
            }
        }
        break;
    }

    case TL_GREEN_BLINK:
    case TL_YELLOW:
        short_red_pending = true;
        break;
    }
}

void traffic_light_init(void)
{
    short_red_pending = false;
    enter_state(TL_RED, HAL_GetTick());
}

void traffic_light_update(void)
{
    uint32_t now = HAL_GetTick();

    if (button_pressed_event() && (mode == TRAFFIC_LIGHT_MODE_1)) {
        on_button(now);
    }

    if (state == TL_GREEN_BLINK &&
        (uint32_t)(now - last_blink_tick) >= BLINK_HALF_MS) {
        last_blink_tick = now;
        led_toggle(LED_GREEN);
    }

    if (elapsed(now) >= state_duration) {
        switch (state) {
        case TL_RED:         enter_state(TL_GREEN, now);       break;
        case TL_GREEN:       enter_state(TL_GREEN_BLINK, now); break;
        case TL_GREEN_BLINK: enter_state(TL_YELLOW, now);      break;
        case TL_YELLOW:      enter_state(TL_RED, now);         break;
        }
    }
```

```
}

const char *traffic_light_state_name(void)
{
    return state_names[state];
}

void traffic_light_set_mode(traffic_light_mode_t new_mode)
{
    mode = new_mode;
    if (mode == TRAFFIC_LIGHT_MODE_2) {
        short_red_pending = false;
    }
}

traffic_light_mode_t traffic_light_get_mode(void)
{
    return mode;
}

bool traffic_light_set_timeout_s(uint32_t seconds)
{
    if ((seconds < MIN_TIMEOUT_S) || (seconds > MAX_TIMEOUT_S)) {
        return false;
    }
    red_ms = seconds * MS_PER_SECOND;
    return true;
}

uint32_t traffic_light_get_timeout_s(void)
{
    return red_ms / MS_PER_SECOND;
}
```

4.11. Главная функция (`main.c`)

```
int main(void)
{
  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick.
*/
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
```

```
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  /* USER CODE BEGIN 2 */
  uart_driver_init();
  console_init();
  led_init();
  button_init();
  traffic_light_init();
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    uart_driver_process();
    button_update();
    traffic_light_update();
    console_update();
  }
  /* USER CODE END 3 */
}
```

5. Вывод

В ходе лабораторной работы был изучен последовательный интерфейс UART и организация системы прерываний микроконтроллера STM32F407. Для стенда SDK-1.1M разработан собственный драйвер UART, который работает напрямую с регистрами `USART_TypeDef` и не использует функции HAL и LL; инициализация UART в конфигураторе (файл `*.ioc`) отключена. Драйвер поддерживает два режима – с использованием прерываний и в режиме опроса – с переключением во время работы программы. В обоих режимах обмен неблокирующий, а принимаемые и передаваемые данные буферизуются в очередях с размером, настраиваемым на этапе компиляции.

Собственная реализация отличается от HAL тем, что не использует дескриптор обмена с состояниями «занят» и задания «передать или принять N байт» с callback-функциями. Вместо этого приём идёт непрерывно в очередь, а передача управляется прерыванием `TXE` до

опустошения очереди. Это исключает ситуацию зависания драйвера в состоянии «занят» при отключении прерываний и убирает активное ожидание, присутствующее в `HAL_UART_Transmit` и `HAL_UART_Receive`. В отличие от LL, где каждая функция лишь оборачивает операцию над одним регистром, драйвер реализует законченную логику: расчёт делителя скорости, обработку ошибок приёма, очереди и переключение режимов. Платой за это является привязка к регистрам семейства STM32F4 (`SR`, `DR`).

На основе драйвера доработана программа «Светофор»: добавлены команды `?`, `set mode`, `set timeout` и `set interrupts`, эхо принимаемых символов и ответы на каждую команду. Прикладная логика (консоль и светофор) не содержит обращений к аппаратуре, все параметры платы собраны в `board_config.h`, а код разбит на независимые модули, пригодные для повторного использования.